

b)

¿Qué es el patrón DTO (Data Transfer Object)? ¿Cómo es su aplicación para el pasaje de información entre capas en el Patrón MVC? ¿Cómo se utiliza el patrón cuando es necesario tener un objeto con atributos de distintas entidades?

Es un patrón de diseño arquitectónico cuyo único propósito es transportar datos entre distintos procesos o capas de una aplicación. Nació originalmente para optimizar aplicaciones distribuidas, reduciendo la cantidad de llamadas remotas al agrupar múltiples datos en un solo envío. Hoy en día, se utiliza masivamente en arquitecturas en capas para desacoplar el modelo de dominio de la interfaz de usuario o las APIs.

Un DTO es un objeto que contiene propiedades (atributos) y sus métodos de acceso (getters/setters), pero **no contiene ningún tipo de lógica de negocio** ni comportamiento.

En una arquitectura MVC moderna, el DTO actúa como el puente de comunicación seguro entre el Controlador y la Vista, protegiendo al Modelo.

Si pasamos las entidades directamente a la vista, acoplaríamos la interfaz gráfica a la base de datos. Si la tabla cambia, la vista se rompe. Además, expondríamos datos vulnerables (como el hash de una contraseña de un usuario).

El flujo de comunicación usando DTOs se estructura de la siguiente manera:

1. **La Vista envía datos:** El cliente completa un formulario y lo envía. El Controlador recibe esta información empaquetada en un **Request DTO** (por ejemplo, CrearUsuarioDTO).
2. **El Controlador procesa:** La capa de servicio recibe el CrearUsuarioDTO, valida la información y mapea esos datos hacia una **Entidad** real (por ejemplo, Usuario).
3. **El Modelo persiste:** La Entidad **Usuario** se guarda en la base de datos.
4. **Respuesta hacia la Vista:** Una vez guardado, el sistema no devuelve la Entidad completa. Mapea la Entidad hacia un **Response DTO** (por ejemplo, UsuarioRespuestaDTO que omite la contraseña y datos internos) y el Controlador se lo envía a la Vista para ser mostrado.

Uno de los usos más potentes del patrón DTO es la **agregación o aplanamiento (flattening)** de datos.

A menudo, una Vista necesita mostrar información que en la base de datos está normalizada y dividida en múltiples tablas (Entidades). En lugar de hacer que la Vista o el Controlador hagan múltiples consultas y armen el rompecabezas, se crea un DTO específico que consolida todo.

Ejemplo de escenario: Se tiene una vista de "Perfil de Empleado". En la base de datos, la información está en tres entidades:

- **Empleado** (Nombre, Apellido, Departamentoid)
- **Departamento** (NombreDepartamento, Ubicacion)
- **Contrato** (Salario, FechaInicio)

La Solución con DTO: Se crea un único objeto llamado **PerfilEmpleadoDTO**. La capa de servicio se encarga de consultar las tres entidades, extraer únicamente los campos relevantes, y llenar este nuevo objeto.

Esta solución aporta eficiencia al darle a la vista lo justo y necesario para renderizar, seguridad por ocultar datos sensibles y mantenibilidad si a futuro la base de datos cambia solo hace falta actualizar el DTO para la vista.

¿Qué es una cookie, cuál es su aplicación en programación Web y cuál es su uso?. Dar ejemplos de uso en sistemas reales. ¿Dónde quedan almacenadas las cookies en una PC? ¿Cuáles son las propiedades que tienen las cookies que pueden ser manipuladas por los programadores? Mostrar una imagen donde se pueden ver las cookies en el navegador web que utiliza.

Una **cookie** es un pequeño archivo de texto que un servidor web envía al navegador del usuario y que este almacena localmente. Su propósito principal es recordar información sobre el usuario a lo largo del tiempo.

El protocolo HTTP, sobre el cual funciona la web, es un protocolo **stateless**. Esto significa que cada vez que se hace clic en un enlace o se recarga una página, el servidor te trata como un usuario completamente nuevo; no tiene memoria de lo que se hizo en la página anterior.

Las cookies resuelven este problema. Permiten **mantener el estado** y crear una sesión continua. Cuando el servidor web devuelve una respuesta, puede incluir una cabecera **Set-Cookie**. El navegador guarda ese dato y, en cada petición posterior a ese mismo servidor, el navegador envía la cookie de vuelta, permitiendo que el servidor reconozca al usuario.

Ejemplos de uso en sistemas reales

1. **Autenticación (Mantener la sesión):** Cuando inicias sesión en Gmail o Facebook, el servidor valida tu contraseña y te envía una cookie con un "token" de sesión único. En tus siguientes clics, esa cookie viaja con tu petición, por lo que no tienes que volver a poner tu contraseña en cada página.

2. **Carritos de compras:** En sitios como Amazon, si navegas como invitado y añadís productos al carrito, estos se guardan en una cookie. Si cierras el navegador y vuelves al día siguiente, el carrito sigue teniendo tus productos.
3. **Personalización:** Guardar preferencias visuales, como si elegiste el "Modo Oscuro" en una página de noticias, o el idioma de preferencia de la interfaz.
4. **Rastreo y Analítica:** Las empresas de publicidad usan cookies de terceros para rastrear tu historial de navegación en diferentes sitios web y mostrarte anuncios personalizados (por ejemplo, cuando buscas zapatillas en Google y luego te aparecen anuncios de zapatillas en Instagram).

Los navegadores modernos almacenan las **cookies** dentro de una **base de datos SQLite** en la carpeta del perfil del usuario.

Propiedades de las cookies manipulables por el programador

Propiedad	Descripción
Name / Value	El nombre de la variable y la información que contiene.
Domain	Define para qué dominio es válida la cookie.
Path	Define las rutas específicas dentro del dominio que pueden leer la cookie.
Expires / Max-Age	Determina la fecha u hora exacta en la que el navegador debe eliminar la cookie. Si no se especifica, es una "cookie de sesión" y se borra al cerrar el navegador.
Secure	Si es true , la cookie solo se enviará a través de conexiones encriptadas HTTPS.
HttpOnly	Si es true , bloquea el acceso a la cookie desde JavaScript (usando document.cookie). Es vital para prevenir ataques XSS (Cross-Site Scripting).
SameSite	Controla si la cookie se envía en peticiones que provienen de otros sitios web (Strict , Lax o None). Ayuda a mitigar ataques CSRF.

Ataques XSS: Es un ataque de inyección de código. Ocurre cuando un atacante logra insertar código malicioso (generalmente JavaScript) en una página web legítima que visitan otros usuarios.

¿Cómo funciona?

1. Imagina un foro donde cualquiera puede dejar un comentario.
2. Un atacante escribe un comentario, pero en lugar de texto normal, escribe código:
<script>enviarDatosAlAtacante(document.cookie);</script>.
3. Si el servidor web no sanitiza ese comentario antes de guardarlo, lo mostrará tal cual.
4. Cuando un usuario normal entra a leer el foro, su navegador ve ese código oculto y lo ejecuta, pensando que es parte de la página. El script roba la cookie de sesión del usuario y se la envía al atacante.

Ataques CSRF: Es una "falsificación de petición en sitios cruzados". Ocurre cuando un atacante engaña al navegador de un usuario autenticado para que ejecute una acción no deseada en un sitio donde ya tiene la sesión iniciada.

¿Cómo funciona?

1. Inicias sesión en tu banco (**banco.com**). Tu navegador guarda la cookie de sesión.
2. Sin cerrar el banco, abris otra pestaña y entras a una página de memes que, sin que lo sepas, está controlada por un atacante (**memes-maliciosos.com**).
3. En esa página de memes hay un botón falso o un código invisible que hace una petición a:
banco.com/transferir?destino=cuentaAtacante&monto=1000.
4. Tu navegador ve que la petición va hacia **banco.com** y automáticamente **adjunta tu cookie de sesión del banco**. El banco recibe la orden, ve que la cookie es válida y transfiere el dinero, creyendo que vos hiciste el clic intencionalmente.

Cookies en Brave

Application

Manifest

Service workers

Storage

WebMCP

Storage

Local storage

Session storage

Extension storage

IndexedDB

Cookies

https://docs.google.c...

https://clients6.goog...

https://contacts.goo...

https://accounts.goo...

Cache storage

Storage buckets

Background services

Back/forward cache

Bounce tracking mitiga...

Notifications

Payment handler

Speculative loads

Push messaging

Device bound sessions

Frames

top

Filter

Only show cookies with an issue

Name	Value	Do...	Path	Expi...	Size	htt...	Secure	Sam...	Part...	Cros...	Prio...
__Secure-1PAPISID	RB7qszljijNJLyBC/A5mzZFoa_lyKdRm...	.go...	/	202...	51		✓				High
__Secure-1PSID	g.a000BgmrLZvtU5jpKnRI6HAeGIC2...	.go...	/	202...	167	✓	✓				High
__Secure-1PSIDCC	AKEyXzUXwKvHZiGf-4wSE_1LPWX5H...	.go...	/	202...	92	✓	✓				High
__Secure-1PSIDTS	sidts-CjEBXMw41R5fw8itaWmOr-U_...	.go...	/	202...	93	✓	✓				High
__Secure-3PAPISID	RB7qszljijNJLyBC/A5mzZFoa_lyKdRm...	.go...	/	202...	51		✓	None			High
__Secure-3PSID	g.a000BgmrLZvtU5jpKnRI6HAeGIC2...	.go...	/	202...	167	✓	✓	None			High
__Secure-3PSIDCC	AKEyXzX45PUS-xaYwKwUbh2bhtqE5...	.go...	/	202...	91	✓	✓	None			High
__Secure-3PSIDTS	sidts-CjEBXMw41R5fw8itaWmOr-U_...	.go...	/	202...	93	✓	✓	None			High
__Secure-BUCKET	CLAC	.go...	/	202...	19	✓	✓				Med...
__Secure-OSID	g.a000BwmrLXcQmsSZe_uLokFiW60...	.doc...	/	202...	166	✓	✓	None			Med...
APISID	ZO_vwO8ZGWMlvx_l/Aqm8HpVN0sF...	.go...	/	202...	40						High
COMPASS	documents=CmlACWuJV5JFzpmMqt...	.doc...	/doc...	202...	337	✓	✓	None			Low
COMPASS	appsfrontendserver=CgAQJ5rB1AYaf...	.doc...	/	202...	423	✓	✓	None			Med...
HSID	A3zci8jE_PWvSCp4j	.go...	/	202...	21	✓					High
NID	534-RISmj2hTPXIFi6lYPxOE1-KBLr7C...	.go...	/	202...	1325	✓	✓	None			Med...
OSID	g.a000BwmrLXcQmsSZe_uLokFiW60...	.doc...	/	202...	157	✓	✓				Med...
OTZ	8725827_68_68_68_	.doc...	/	202...	21		✓				Med...
SAPISID	RB7qszljijNJLyBC/A5mzZFoa_lyKdRm...	.go...	/	202...	41		✓				High
SEARCH_SAMESITE	CgQIsqEB	.go...	/	202...	23			Strict			Med...
SID	g.a000BgmrLZvtU5jpKnRI6HAeGIC2...	.go...	/	202...	156						High
SIDCC	AKEyXzUmhPsUhi_DmHpApG0J8d2jS...	.go...	/	202...	80						High
SSID	Acn7M7YnASlCfkT5B	.go...	/	202...	21	✓	✓				High

No cookie selected

Select a cookie to preview its value